

A Unified Vision for Manipulating Tuple-like Objects

Document: P0311R0
Date: 2016-03-18
Project: ISO/IEC JTC1 SC22 WG21 Programming Language C++
Audience: Evolution Working Group
Author: Matthew Woehlke (mwoehlke.floss@gmail.com)

Abstract

There is much activity and discussion surrounding tuple-like objects, with many features being requested and many papers submitted or planned. It is important that we establish a plan for where we are going that takes into account future directions in order to avoid overcomplicating the language or painting ourselves into a corner of incompatible features.

Contents

- [Abstract](#)
- [Background](#)
- [Preface](#)
- [Definitions](#)
- [Access](#)
 - [An operator-like alternative](#)
- [Generalized Unpacking](#)
- [Unification of Unpacking](#)
- [Slicing](#)
- [Pack Generation, Revisited](#)
- [Summary](#)
- [Acknowledgments](#)
- [Footnotes](#)
- [References](#)

Background

At the 2016 Jacksonville meeting, [P0144](#) was discussed for the second time. Reception was generally positive, but some issues remained to be addressed. It seems quite clear that this is a desired direction for C++. Unfortunately, [P0197](#), which was scheduled for presentation and has some impact on the direction which [P0144](#) is following, was skipped due to time constraints.

Discussion on the `std-proposals` forum often brings up the desire to extend use of "tuple-like" objects to contexts other than name binding (i.e. [P0144](#)). There is also significant and related discussion on improving the usability of parameter packs. We feel that several of these areas are closely related and warrant the formation of a concrete and unified vision for future direction.

Preface

We present several suggestions in this paper, along with accompanying syntax. We must stress *emphatically* that this paper is not intended to propose any features in and of itself (that will come later). Rather, we wish to outline several areas of anticipated future development which we feel need to be explored and, in particular, considered by other proposals being put forward, especially [P0144](#). While some brief statements are made as to our choices, we urge the reader to keep in mind that syntax shown is used in this context only as a tool to communicate examples of the future feature space, and not to get hung up on minor quibbles. In particular, our most immediate concern is for unification of implementation details related to unpacking and customization points of the same. While we feel also that similar considerations are important with respect to parameter packs, that feature space is less mature, and accordingly the need for a consolidated direction is less urgent, if no less real.

Definitions

A "tuple like" is any object consisting of one or (usually) more orthogonal values (in mathematical notation, a "product type").

The canonical example is `std::tuple`, but other examples include `std::array` or similar fixed-size vector types and most aggregates, as well as some non-aggregate user types including ones with no public NSDM's[\[1\]](#).

"Unpacking" refers to the conversion of a tuple-like object into its component parts. This includes both name-binding unpacking (i.e. [P0144](#)) and "generalized unpacking" where the components are used in a non-binding context; for example, as values in a function parameter list. Name-binding unpacking is also called "structured binding" and, historically, "assignment unpacking". We prefer the term "name-binding unpacking" as it does not call into question issues of "true assignment" versus aliasing where [P0144](#) specifically desires to avoid certain overheads, and the use of "unpacking" serves to connect two closely related concepts.

Access

One of the active questions around [P0144](#) regards the customization point. We feel strongly that the customization point for name-binding unpacking should be the same as used by generalized unpacking and by existing and proposed utility functions (e.g. `std::apply` and `std::make_from_tuple`) that act on tuple-like objects. This is important for the sake of consistency; these operations are extremely similar, and using different customization points will likely result in confusion and teaching difficulty.

That said, we feel less strongly about the exact nature of those customization points, providing that those points which are eventually used provide satisfactory backwards compatibility.

At present, these customization points are:

```
get<N>(T):  
    Access the N'th value of the tuple-like, where 0 < N < tuple_size(T).  
  
constexpr tuple_size(T):  
    Returns the size of (i.e. number of elements in) the tuple-like.
```

An operator-like alternative

Some concerns were expressed that overloading on `get<N>(T)` is not appropriate due to its use for other operations that are not related to tuple-like objects. One alternative might be to implement a new operator type:

```
operator get(auto& tuple, constexpr size_t i);  
constexpr operator sizeof<T>();
```

It may be reasonable or even desirable to restrict access of these operators to either explicit spelling or use of dedicated syntax:

```
MyTupleLike t;  
  
[0]t; // operator get  
sizeof...(t); // operator sizeof  
  
auto [x, y] = t; // both, via name-binding unpacking, case 2
```

We should note that, while there are some strong feelings on these topics, we do not feel that any particular resolution is critical for any of the directions we are exploring. In this area, we feel only that a consistent and clear direction is important.

(Types have been elided in the above examples, as they are not crucial to the discussion.)

Generalized Unpacking

Generalized unpacking is the conversion of a tuple-like to a "value sequence", in the manner of Python's `*` operator, such that the resulting sequence may be used in any place that a comma separated sequence may be used. While function parameter lists is the canonical example, this would also include braced initializer lists. Following [discussion](#) on the `std-proposals` forum, we believe that the most reasonable and useful mechanism of accomplishing this is to provide a mechanism whereby a tuple-like may be converted into a parameter pack. Much as in the name-binding unpacking case, there is a logical code transformation that can be applied for this purpose, by placing the tuple-like into a temporary (where necessary, i.e. if the tuple-like is an expression rather than already a named variable) and taking the parameter pack to be `get<0>(__t), get<1>(__t), ...`. This extends the usable scope to anywhere a fold expression may be used.

We are aware of at least three possible mechanisms for implementing generalized unpacking. One option is to employ a new syntax to perform this operation directly. Another is to make multiple return values, treated as parameter packs, first class citizens of the language. A third is to create a parameter pack "generator". The latter two options make it possible to write a function (which might reasonably be named `std::unpack`) that is equivalent to the former.

Several possible syntaxes have been proposed, including postfix operator `~`. Our preference, however, is prefix operator `[:]` (for reasons that will be — very briefly — shown later, in [Slicing](#)), which we will use here, always bearing in mind that this is strictly for demonstrative purposes. For example:

```
struct { double x, y; } point = ...;
auto h = std::hypot([:]point...);
```

The addition of such a feature, regardless of its form[\[2\]](#), would obviate most (though perhaps not all) use cases for `std::apply` and `std::make_from_tuple`. It would also permit trivial conversions between different "simple" types which are distinct but layout compatible, by unpacking the first type into a braced initializer list used to construct the second. We believe that this feature will be at least as important and useful as name-binding unpacking.

Unification of Unpacking

Possibly the most important aspect of [P0197](#) in our opinion is the provision for a single, unified mechanism for unpacking, whether in the name-binding or generalized senses. The critical aspect of [P0197](#), and the one that we feel strongly needs to be considered by [P0144](#), is providing implicit general tuple-like access to simple data structures. In particular, we feel that it would be a travesty for name-binding unpacking and generalized unpacking to use different customization points or to otherwise behave differently when used in ways where intuition strongly expects equivalent behavior. In particular, we feel strongly that, for a tuple-like type having a default destructor, the following should be equivalent (after optimizations):

```
auto [x, y] = t;
auto [x, y] = {[:]t...};
```

(This illustrates a need to be careful with lifetime semantics; in particular, unpacking should likely either extend lifetime when used in a braced initializer list, or should explicitly create value copies in such case. The former would make the above equivalent for *any* tuple-like, while the latter may be useful for separating lifetime of the tuple-like and its components. We do not recommend a direction at this time, although this is likely to be of relevance when considering a language solution versus a "library" solution.)

It should be noted that [P0197](#) would provide a modest enhancement to name-binding unpacking. Where [P0144](#) limits itself to "flat" classes, [P0197](#) would extend implicit tuple-like access to all classes which:

- Contain no non-public NSDM's
- Contain no members of union type
- Have no virtual[\[3\]](#) and/or non-public[\[4\]](#) base classes
- Have no base classes which do not also meet the preceding eligibility criteria

While it would not be a catastrophic loss if non-"flat" classes were not supported, we do feel that it would be most unfortunate if we are not able — eventually — to rely on this implicit access to implement name-binding unpacking, and accordingly to eliminate [P0144](#) case 3. In addition to consistency, we feel that this is important for the sake of simplicity, as it eliminates a special case from name-binding unpacking. We are confident that the performance issues (that is, the understanding that case 3 represents name aliasing and neither consumes storage beyond that required for the tuple-like itself nor adds any access indirection) can be satisfactorily addressed through compiler optimization, keeping in mind of course that the implementations of the "get" function (however we ultimately spell it) are inline in these instances.

The problem that arises from this approach is bitfield members. At the 2016 Jacksonville meeting, at least one individual expressed a strong opinion that providing read/write access to bitfield members via name-binding unpacking is a "must have" feature. We encourage giving serious consideration to the true importance of this feature, and to ways that this could be addressed in a way that does not require special casing. (In particular, we note that the general ability to have a reference to a bitfield — likely through some new library type — seems at least as interesting as being able to name-bind to a component of such type of a tuple-like.)

Slicing

In our earlier discussion on [Access](#), we mentioned syntax for accessing specific elements of a tuple-like. While the need to access individual elements is obvious and clearly does not require a syntactic solution (we already have `std::get<N>`), another desire that comes up often is the ability to slice a tuple-like; e.g. to strip the first element or take only the first N elements.

We chose `[:]` because it naturally extends to slicing, but various possible solutions have been suggested, including pack generators (which would offer significant expressive power). More importantly, since we recommend that generalized unpacking convert a tuple-like to a parameter pack, it makes sense that a syntax for slicing tuple-likes should also work on parameter packs directly. In addition to the advantages for tuple-likes, this enables simple and powerful transformations for

variadic templates, thus satisfying another important contemporary use case. In particular, we can now write recursive variadic template functions like:

```
void print_each() {} // sentinel

template <typename... T>
void print_each(T... values)
{
    print_one([0]values);
    print_each([1:]values);
}
```

This is a fairly trivial example that previously could be written by breaking the complete pack into a separately named head argument and tail pack. This, however, merely scratches the surface. One could imagine implementing a `constexpr` divide-and-conquer sort algorithm using slicing to trivially split the incoming parameter pack in half. Many other examples which can be readily implemented with slicing but would be difficult and/or expensive to implement otherwise can be imagined.

Pack Generation, Revisited

Parameter pack generation is, in general, an interesting feature. Suggested example uses include generating an integer list[\[5\]](#), a type list, and performing various manipulations on parameter packs. While such manipulations could include slicing and reversing, we note that these operations appear to rely on a syntactic mechanism for extracting a single element from a pack (reference is made to [N4235](#)). We also wonder if slicing operations implemented in this manner would perform satisfactorily compared to syntactic slicing.

Consequently, we still need a syntax for indexed access of parameter pack elements. This in turn allows us to apply the previous argument in reverse; namely, why not select a syntax that is non-ambiguous, easily extended to slicing, and may be applied also to tuple-likes? This is a point that we feel is worth serious consideration as we consider what direction generalized unpacking should take.

Summary

Previous discussions — both in EWG and on the `std-proposals` forum — suggest a strong desire by the C++ community to move the language in a direction that blurs the line between containers and their contained value sequences, making it easy to move from one to the other, as is often found in other languages (e.g. Python). At the same time, there are a number of proposals either published or in the works to simplify working with parameter packs. Moreover, due to the significant utility of unpacking and otherwise working with tuple-like objects as parameter packs, these areas are closely related and to some extent overlap.

We have observed recently that "complexity" is a frequent complaint made against C++, especially that it is "hard to teach". As we consider features to simplify working with tuple-like objects and/or parameter packs, we feel it is of utmost importance to establish and adhere to a consistent vision of these functions, in terms of both syntax and function. We specifically urge that name-binding unpacking would carefully consider customization points[\[6\]](#) and the future possibility of implicit tuple-like access (see especially [P0197](#)) and generalized unpacking in order to work toward[\[7\]](#) a common mechanism for both that would eliminate special case rules specific to the individual features. We also urge the committee to consider these issues and how such features relate (or can be made to relate) to tuple-like objects in order to maximize consistency of operations on both object types, and we urge authors working on such proposals to do likewise. Finally, we strongly encourage any authors working in this realm to maintain communication in order to reduce the dangers of competing, incompatible proposals and to maximize our ability as a community to pursue a well considered, consistent, and maximally functional direction.

By offering a glimpse at where we might be going, we hope we have demonstrated the importance of keeping the future in mind while developing new and exciting features today. We especially hope we have demonstrated the importance of considering the direction proposed by [P0197](#) (implicit tuple-like access for "simple" types) in light of [P0144](#) (name-binding unpacking) in order to maintain consistency and simplicity of specification in order to maximize the ability of users to understand the operation of these features and to use them in a sensible manner.

Acknowledgments

We would like to thank the authors of [P0144](#), for obvious reasons. We would like to thank Mike Spertus and Daveed Vandevoorde for sharing a "preview" of their respective works-in-progress in the area of parameter packs. We would like to thank Daniel Frey for his own work on parameter packs, which also forced us to consider the defense our own preferences more strenuously than had been done before. As always, we would also like to thank everyone that has shared their thoughts and ideas on these issues, both in person at the 2016 Jacksonville meeting and on `std-proposals`.

Footnotes

- [1] [QVector3D](#) comes to mind as an example of a user type which is — or at least, ought to be — tuple-like but has no public data members.
- [2] The form that unpacking takes is not entirely uninteresting, however such discussion is not in scope for this paper.
- [3] While present in the initial revision of [P0197](#), this restriction is not seen in [P0144](#), and upon further consideration, may be unnecessary.
- [4] This could probably be relaxed to non-public *and non-empty* base classes, if desired.
- [5] The purely template implementation of `std::integer_sequence` is extremely expensive, to the point that many compilers are providing implementations based on compiler intrinsics. Parameter pack generators have the potential to provide a satisfactory implementation without such intrinsics.
- [6] It is our understanding that the committee and the authors of [P0144](#) are well aware of the strong feelings surrounding customization points and *are* giving them serious consideration. We wish to take this opportunity to thank and commend them for these efforts.
- [7] We would like to reiterate that we have no objection to special case handling of "implicitly tuple-like" types in the short term, especially if it means name-binding unpacking is available in C++17, *provided* there is a long term migration route that would allow this special case to be replaced with more generalized functionality.

References

- (Discussion) `std::invoke` and unpacking tuple-like type instances
<https://groups.google.com/a/isocpp.org/d/msg/std-proposals/PghsmqN1cAw/0Q1V-22lFAAJ>
- (Discussion) Unpacking tuples to value sequences
<https://groups.google.com/a/isocpp.org/d/msg/std-proposals/KW2FcaRAasc/Xc9lxRB1FwAJ>
- (Discussion) Extracting tuples out of a tuple
https://groups.google.com/a/isocpp.org/d/msg/std-proposals/-81BeWT5DCA/Xs8uPY_zHgAJ
- (Discussion) Improve fundamentals of parameter packs
<https://groups.google.com/a/isocpp.org/d/msg/std-proposals/ajLcDl8GbpA/woiAbredAwAJ>
- [N4235](#) Selecting from Parameter Packs
<http://wg21.link/n4235>
- [P0144](#) Structured Bindings
<http://wg21.link/p0144>
- [P0197](#) Default Tuple-like Access
<http://wg21.link/p0197>